

Exercise 3b

Phase 3 -- Multiple interacting objects

Most real OOP begins here.

Cleaned up and commented version of Exercise 3a

Exercise 3: Communication chain

	src	enc	ch	dec	rx	
---->	Source	Encoder	Channel	Decoder	Receiver	----
text	bits	coded	noisy	decoded	BER, rtxt	

State:	text	G	PbE	H	noisy, decoded
Methods:	generate() set_text() set_LSBfirst()	encode()	transmit() set_PbE() get_PbE()	decode() syndrome()	received_text() BERchan() BERinfo()

```
import numpy as np
rng = np.random.default_rng()
```

Source Class

Create:

```
class Source:
```

Internal state:

text
LSBfirst

Methods:

set_text()
set_LSBfirst()
generate()

```
class Source:
    """
    Converts ASCII characters to 8-bit binary with LSB first (LSBfirst=1, default) or MSB first.
    Output is character-by-character bitstream
    """

    def __init__(self, text, LSBfirst=1):
        if len(text) < 1:
            raise ValueError("text must have length > 1")
        self.text = text
        self.LSBfirst = LSBfirst

    def set_text(self, text):
        self.text = text

    def set_LSBfirst(self, LSBfirst):
        self.LSBfirst = LSBfirst

    def generate(self):
        bitsperchar = 8
        textnum = np.array([ord(c) for c in self.text]) # convert ASCII to numeric
```

```

if self.LSBfirst == 1:
    p2 = np.power(2.0, np.arange(0, -bitsperchar, -1))
else:
    p2 = np.power(2.0, 1 + np.arange(-bitsperchar, 0))
B = np.array(np.mod(np.array(np.floor(np.outer(textnum, p2))), int), 2), np.int8)
bits = np.reshape(B, B.size)
return bits

```

Encoder Class

Create:

```
class Encoder:
```

Internal state:

```
G      # Generator matrix
```

Methods:

```
encode()
```

```

class Encoder:
    """
    Uses linear binary systematic (n,k) Hamming-style code, characterized by generator matrix G.
    Input bits are broken up into blocks of length k and encoded into blocks of length n.
    """

    def __init__(self, G):
        self.G = np.array(G, int)
        self.k, self.n = np.shape(self.G)

    def encode(self, bits):
        N = int(np.ceil(len(bits)/self.k))
        bits_pad = np.zeros(N*self.k, int)
        bits_pad[:len(bits)] = bits
        u = np.reshape(bits_pad, (-1, self.k)) # split bits string into blocks of length k
        B = np.mod(u@self.G, 2) # encoding using generator matrix G
        return np.reshape(B, (1, -1)).flatten()

```

Channel Class

Create:

```
class Channel:
```

Internal state:

```
PbE      # Probability of bit error
```

Methods:

```
transmit()
```

```
get_PbE()
```

```
set_PbE()
```

```

class Channel:
    """
    Binary symmetric channel (BSC) with uniformly distributed probability of bit error (PbE)
    """

    def __init__(self, PbE):
        if (PbE < 0) or (PbE > 1):

```

```

        raise ValueError("PbE must be in range 0...1.0")
    self.PbE = PbE

def get_PbE(self):
    return self.PbE

def set_PbE(self, PbE):
    if (PbE < 0) or (PbE > 1):
        raise ValueError("PbE must be in range 0...1.0")
    self.PbE = PbE

def transmit(self, binary_in):
    L = len(binary_in)
    ix = np.where(rng.random(L) <= self.PbE)
    out = binary_in.copy()
    out[ix] = np.mod(out[ix] + 1, 2)    # flip error bits
    return out

```

Decoder Class

Create:

```
class Decoder:
```

Internal state:

```
H    # Parity check matrix
```

Methods:

```
decode()
```

```
syndrome()
```

```
class Decoder:
```

```
    """
```

```

    Linear binary systematic (n,k) single error correcting code defined by parity chck matrix H.
    Received noisy bits are broken up into blocks of length n.
    Each non-zero syndrome  $s=rH^T$  corresponds to a single error in a block of length n.
    After error correction the first k bits in each block are concatenated to form the decoded bit string.
    """

```

```

def __init__(self, H):
    self.H = np.array(H, int)
    self.nk, self.n = np.shape(self.H)
    self.map = self.__syn2bit()

```

```

def __syn2bit(self):    # private function, syndrome to bit map
    syn = self.H.T@np.array(2*np.arange(self.nk-1,-1,-1))    # syndrome in decimal
    map = -np.ones(self.n+1, int)
    for i in range(self.n):
        map[syn[i]] = i
    return map

```

```

def syndrome(self, noisy):    # compute syndromes
    N = int(np.ceil(len(noisy)/self.n))
    noisy_pad = np.zeros(N*self.n, int)
    noisy_pad[:len(noisy)] = np.array(noisy, int)
    v = np.reshape(noisy_pad, (-1, self.n))    # split noisy string into blocks of length n
    return np.mod(v@self.H.T, 2)

```

```
def decode(self, noisy):    # decode noisy bit stream
```

```

N = int(np.ceil(len(noisy)/self.n))
noisy_pad = np.zeros(N*self.n, int)
noisy_pad[:len(noisy)] = np.array(noisy, int)
S = self.syndrome(noisy_pad)
ss = S@np.array(2**np.arange(self.nk-1,-1,-1))
ixe = []
for i in range(len(ss)):
    if ss[i]>0:
        ixel.append(self.n*i + self.map[ss[i]])
corrected = np.copy(noisy_pad)
corrected[ixel] = np.mod(corrected[ixel] + 1, 2)
D = np.reshape(corrected, (-1, self.n))
decoded = np.reshape(D[:,self.n-self.nk], (1, -1)).flatten()
return decoded

```

Receiver Class

Create:

```
class Receiver:
```

Internal state:

```
noisy
decoded
```

Methods:

```
BERchan()
BERinfo()
received_text()
```

```

class Receiver:
    """
    Computes channel bit error rate (BERchan) from coded and noisy bit streams.
    Computes information bit error rate (BERinfo) from bits and decoded bit streams.
    Converts decoded bit stream back to ASCII characters
    """

    def __init__(self, noisy, decoded):
        self.noisy = noisy
        self.decoded = decoded

    def BERchan(self, bits):    # channel bit error rate (fraction of flipped noisy bits)
        L = min(len(bits), len(self.noisy))
        BER = np.where(bits[:L]!=self.noisy[:L])[0].size/L
        return BER

    def BERinfo(self, bits):    # information bit error rate (fraction of incorrectly decoded bits)
        L = min(len(bits), len(self.decoded))
        BER = np.where(bits[:L]!=self.decoded[:L])[0].size/L
        return BER

    def received_text(self, LSBfirst=1):
        bitsperchar = 8
        dnb = self.decoded[0:bitsperchar*int(len(self.decoded)/bitsperchar)]
            # make multiple of bitsperchar long
        B = np.array(np.reshape(dnb, (int(len(dnb)/bitsperchar), bitsperchar)), int)
        if LSBfirst == 1:
            p2 = np.power(2, np.arange(0, bitsperchar))
        else:

```

```

        p2 = np.power(2, np.arange(bitsperchar, 0, -1) - 1)
        textnum = np.array(np.dot(B, p2))
        #rxtxt = ''.join(chr(n) for n in textnum)
        rxtxt = ''.join(chr(n%128) for n in textnum)    # convert to 7-bit ASCII
        return rxtxt

# Test sentence and PbE
src = Source("The quick brown fox jumps over the lazy dog 0123456789!", LSBfirst=1)
PbE = 0.01
ch = Channel(PbE)

# Test encoder 1
G1 = [[1,0,0,0,1,1,1],[0,1,0,0,1,1,0],[0,0,1,0,1,0,1],[0,0,0,1,0,1,1]]
enc1 = Encoder(G1)
H1 = [[1,1,1,0,1,0,0],[1,1,0,1,0,1,0],[1,0,1,1,0,0,1]]
dec1 = Decoder(H1)
coded1 = enc1.encode(src.generate())
noisy1 = ch.transmit(coded1)
decoded1 = dec1.decode(noisy1)
rx1 = Receiver(noisy1, decoded1)
print(f'BER1chan: {rx1.BERchan(coded1):0.4f}')
print(f'BER1info: {rx1.BERinfo(src.generate()):0.4f}')
print(rx1.received_text(src.LSBfirst))

BER1chan: 0.0078
BER1info: 0.0000
The quick brown fox jumps over the lazy dog 0123456789!

# Test encoder 2
G2 = [[1,0,0,0,0,0,0,0,0,0,1,1,1,1],
       [0,1,0,0,0,0,0,0,0,0,1,1,1,0],
       [0,0,1,0,0,0,0,0,0,0,1,1,0,1],
       [0,0,0,1,0,0,0,0,0,0,1,1,0,0],
       [0,0,0,0,1,0,0,0,0,0,1,0,1,1],
       [0,0,0,0,0,1,0,0,0,0,1,0,1,0],
       [0,0,0,0,0,0,1,0,0,0,1,0,0,1],
       [0,0,0,0,0,0,0,1,0,0,0,1,1,1],
       [0,0,0,0,0,0,0,0,1,0,0,0,1,1,0],
       [0,0,0,0,0,0,0,0,0,1,0,0,1,0,1],
       [0,0,0,0,0,0,0,0,0,0,1,0,0,1,1]]
enc2 = Encoder(G2)
H2 = [[1,1,1,1,1,1,1,0,0,0,0,1,0,0,0],
       [1,1,1,1,0,0,0,1,1,1,0,0,1,0,0],
       [1,1,0,0,1,1,0,1,1,0,1,0,0,1,0],
       [1,0,1,0,1,0,1,1,0,1,1,0,0,0,1]]
dec2 = Decoder(H2)
coded2 = enc2.encode(src.generate())
noisy2 = ch.transmit(coded2)
decoded2 = dec2.decode(noisy2)
rx2 = Receiver(noisy2, decoded2)
print(f'BER2chan: {rx2.BERchan(coded2):0.4f}')
print(f'BER2info: {rx2.BERinfo(src.generate()):0.4f}')
print(rx2.received_text(src.LSBfirst))

BER2chan: 0.0117
BER2info: 0.0000
The quick brown fox jumps over the lazy dog 0123456789!

```